

Tervezési minták

Egy programozási feladat megoldása gyorsabb, az előállított program biztonságosabb, ha a megoldást korábbi, hasonló feladatok megoldásainál bevált minták alapján állítjuk elő. A szoftvertechnológiában az ilyen programozási mintáknak számos csoportja jött létre, mint például az algoritmus minták (programozási tételek), architektúra minták, vagy a tervezési minták (tervminták, design patterns).

A tervezési minták nem a programok helyességét garantálják, mint ahogy az algoritmus minták teszik, hanem az olyan nem funkcionális követelmények kielégítésében játszanak szerepet, mint az újrafelhasználhatóság, hatékonyság, biztonság, módosíthatóság, és – nem utolsósorban – a SOLID tervezési elveknek való megfelelés¹.

A tervezési minták között klasszikusnak számít az a 23 darab minta, amit 1995-ben írt le Gamma, Helm, Johnson, és Vlissides, és amelyeket a GoF (Gang of Fours), tervmintáiként szoktuk emlegetni. Ezeket a tervezési mintákat három csoportba szokás sorolni: létrehozási, szerkezeti, és viselkedési mintákra.

A tervezési minták többségének előnye igazán akkor mutatkozik, ha nagyobb méretű alkalmazásokat kell készítenünk, ráadásul olyanokat, ahol számítani kell annak karbantartására, bővítésére.

Nézzünk meg most néhány egyszerűbb (kisebb programok készítésénél is alkalmazható) mintát.

¹ Az objektumelvű szoftvertervezés fontos alapelveit fogalmazzák meg az ún. **SOLID elvek**:

1. **Egyszeres felelősség elve (Single responsibility)**

Egy objektumnak egy jól meghatározható felelősségi köre legyen, amelyet egy vagy több metódusa biztosít. Minden osztály, illetve metódus a megoldás egy jól meghatározott részéért feleljen, azaz ne hozzunk létre ún. „mindenható” objektumokat/metódusokat, de ne is osszuk meg ugyanazt a felelősséget több különböző elem között (különösen elkerülve azt, hogy ugyanazt a funkcionalitást redundáns módon lássák el).

2. **Nyílt/Zárt elv (Open/Closed)**

Egy előállított megoldást (modell és kód) lehessen módosítani, de csak úgy, hogy a meglévő elemeken (feltéve persze, hogy azok jók) ne kelljen változtatni: meglévő osztályok mellé lehessen újakat felvenni, egy osztály metódusait újakkal kibővíteni, de egy már tesztelt metódus kódján ne kelljen változtatni.

3. **Liskov helyettesítési elv (Liskov substitution)**

Ha egy ősosztálybeli metódust, amelyik paraméterének típusa egy ősosztály, és a meghívásakor ennek a paraméternek akár ősosztálybeli, akár egy alosztálybeli objektum hivatkozását adjuk, akkor metódusnak mindkét esetben a megfelelő, az elvárt (egymástól sokszor eltérő) működést kell mutatnia.

4. **Interfész elválasztási elv (Interface segregation)**

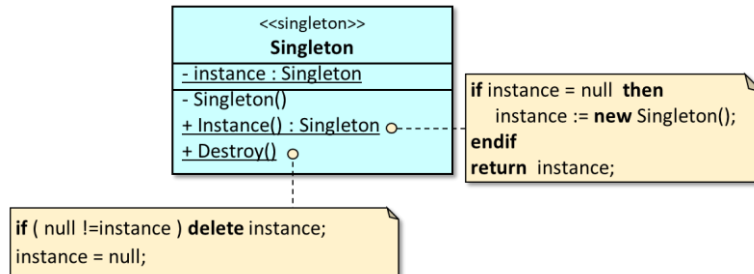
Vezessünk be interfészt (vagy absztrakt ősosztályt) egy-egy adott feladatkör ellátására tervezett, alternatív szolgáltatásokat nyújtó osztályok fölé, amelyet ezek az osztályok megvalósítanak (vagy abból származnak). Minden kliens, amely használja ezen osztályok szolgáltatásait, csak azokat a metódusokat lássa, amelyeket az interfész (absztrakt osztály) publikál.

5. **Függőség megfordítási elv (Dependency inversion)**

Egy magas szintű (kliens) osztály ne függjen közvetlenül alacsony szintű implementációs részleteket kidolgozó osztályoktól, csak interfészekről. Ezáltal megfordul a függőség, a kliens határozza meg az interfész létrehozásával, hogy milyen felületű alacsony absztrakciós szintű osztályokra van szüksége.

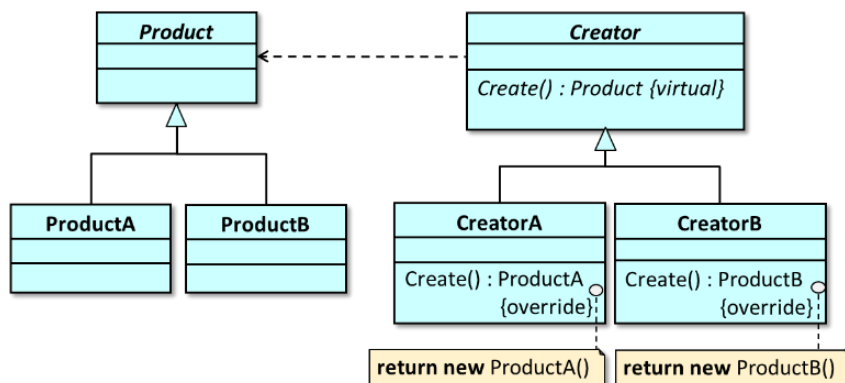
Egyke (Singleton – létrehozási tervezési minta)

Az egyke (singleton) minta biztosítja, hogy egy osztálynak legfeljebb egy példányát, egyetlen objektumát lehessen példányosítani függetlenül a példányosítási kérések számától. (Ennek létjogosultságára elég azt a szélsőséges esetet felhozni, amikor a szóban forgó osztálynak egyáltalán nincsenek adattagjai, azaz bármennyi objektumot hozunk belőle létre, azok tartalmukban nem különböznenek egymástól, de külön-külön memóriát foglalnak.)



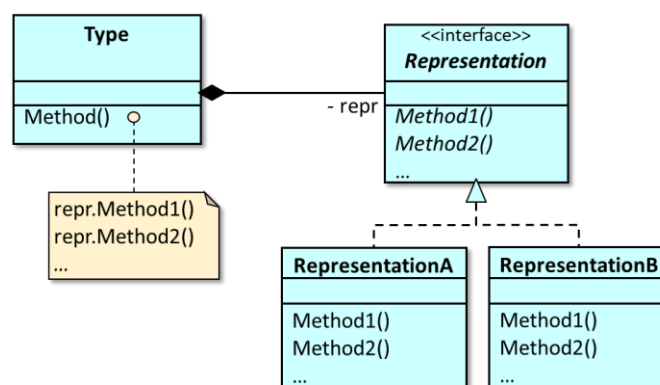
Gyártófüggvény (Factory Method – létrehozási tervezési minta)

Egy ősz osztály alosztályainak objektumait példányosító metódust vagy metódusok csoportját nevezzük így. Egyszerűbb esetben egy ilyen metódust készítünk, amelyik az ősz osztály olyan osztályszintű metódusa lesz, amelyik a neki eljuttatott paraméterek alapján példányosítja valamelyik alosztályt, és adja vissza a létrehozott objektum hivatkozását. Bonyolultabb esetben bevezethetünk minden alosztályhoz egy saját gyártófüggvényt (Create()), és ezeket az eredeti osztályokéhoz hasonló hierarchiába szervezhetjük úgy, hogy az egyes gyártófüggvények külön-külön úgynevezett gyártóosztályok metódusai legyenek, amely gyártóosztályok ősz osztálya (Creator) a gyártófüggvények közös interfészét határozza meg.



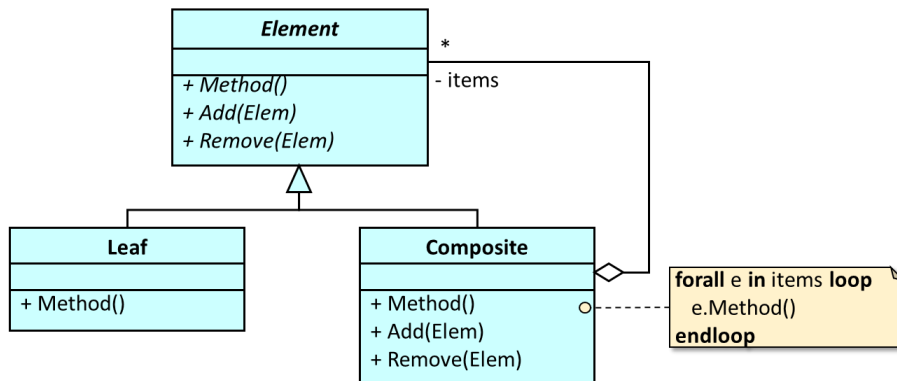
Híd (Bridge – szerkezeti tervezési minta)

Egy típus reprezentációját leválasztjuk a típust biztosító osztályról azért, hogy annak különféle reprezentációi közül rugalmasan, futási időben választhassuk ki a legalkalmasabbat.



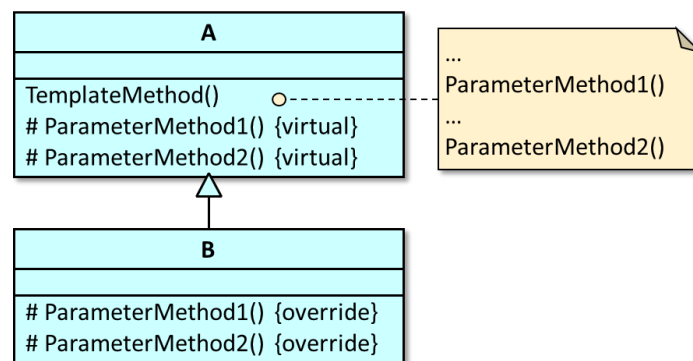
Összetétel (Composite – szerkezeti tervezési minta)

Objektumok fastruktúrába rendezéséhez ajánlják különösen akkor, ha rész-egész (gyerek-szülő) viszonyokat kell kezelni. A módszer révén a levélcúcsokat és a részfákat egységesen kezelhetjük.



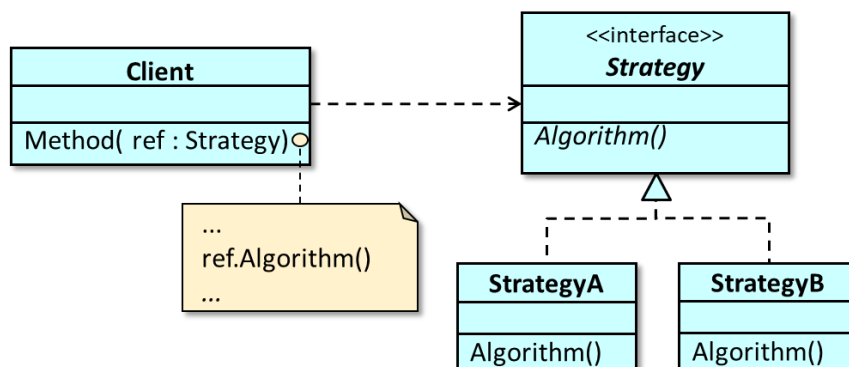
Sablonfüggvény (Template Method – viselkedési tervezési minta)

Egy tevékenységet (sablonfüggvényt) egy őssztály metódusaként definiálunk úgy, hogy annak speciális, alosztálytól függő részeit csak egy-egy virtuális metódus-hívás jelezze (sablon-paraméterek), majd ezen metódusokat az alosztályokban az ott elvárt módon definiáljuk felül.



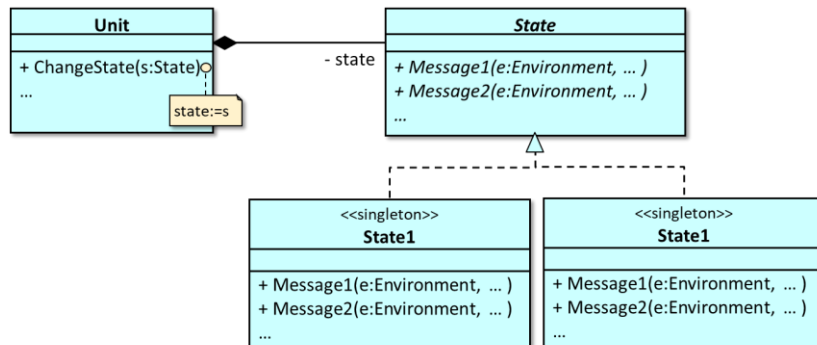
Stratégia (Strategy – viselkedési tervezési minta)

Tevékenységek (algoritmusok, stratégiák) családját olyan osztályok metódusaként vezetünk be, amelyek egy közös interfészt valósítanak meg azért, hogy futási időben lehessen kiválasztani azt, amelyiket egy másik osztály (Client) egy metódusa használni akar. A felhasználni kívánt stratégiát (az `Algorithm()` metódust) hordozó objektumot a kliens objektum megkaphatja aggregálással (objektum befecskendezéssel), vagy metódusának paraméterként való átadásával (lásd itt).



Állapot (State – viselkedési tervezési minta)

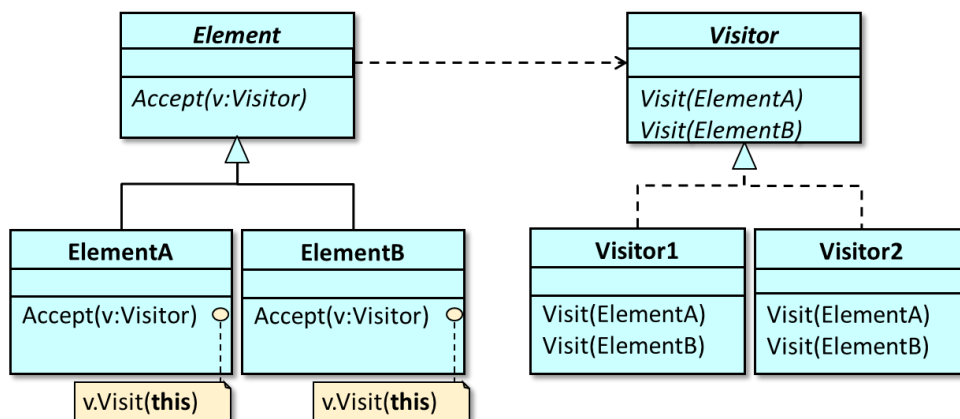
Amikor egy objektum metódusának az objektum állapotától függően kell viselkednie, akkor ahelyett, hogy ezen metódusba az objektum állapotától függő elágazást építenénk, a viselkedést kiszervezzük az objektum állapotait megtestesítő közös interfésszel rendelkező (többnyire egyke) objektumokba. A közös interfésze specifikálja az állapotfüggő metódusokat, míg az eredeti (Unit típusú) objektum rendelkezik az állapotváltoztató metódusokkal.



Látogató (Visitor – viselkedési tervezési minta)

Amikor két objektum család egy-egy objektuma közötti interakcióra kerül sor, akkor ezt a működést a látogató tervminta alapján érdemes szervezni. Az Accept() működése a saját osztályán kívül attól is függ, hogy a paramétere egy osztály melyik alosztályának objektumára hivatkozik, és nem akarjuk, hogy ez a függőség egy elágazás (vagy ugyanolyan szerkezetű elágazások) formájában jelenjen meg a kódban.²

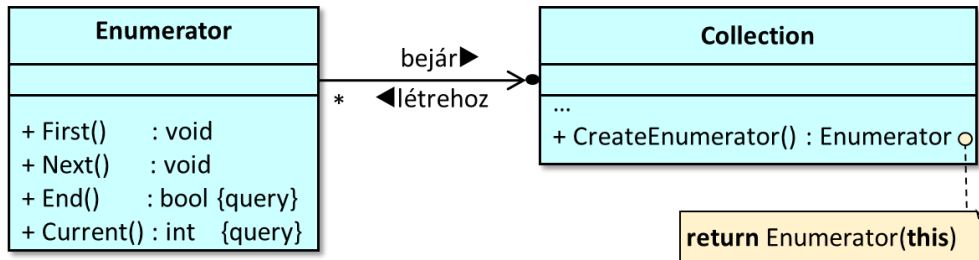
Megfigyelhető, hogy az Accept() metódusok törzse alakilag azonos, de eltér bennük a this típusa. Ezért amikor sor kerül a v.Visit(this) hívására, akkor a Visit() több változata közül az fut majd le, amelyet egyfelől a v paraméternek konkrét osztálya, másfelől a paraméteréből adott this konkrét osztálya határoz meg.



² Ez a minta a programozási nyelvek két fontos nyelvi tulajdonságára támaszkodik: a futási idejű (dinamikus altípusos) polimorfizmusra, és a parametrikus polimorfizmusra (többalakúságra). A dinamikus altípusos polimorfizmus attól jön létre, hogy a származtatás során egy virtuális metódust felüldefiniálunk; a parametrikus polimorfizmus pedig úgy, hogy ugyanolyan néven, de eltérő szignatúrával definiálunk függvényeket.

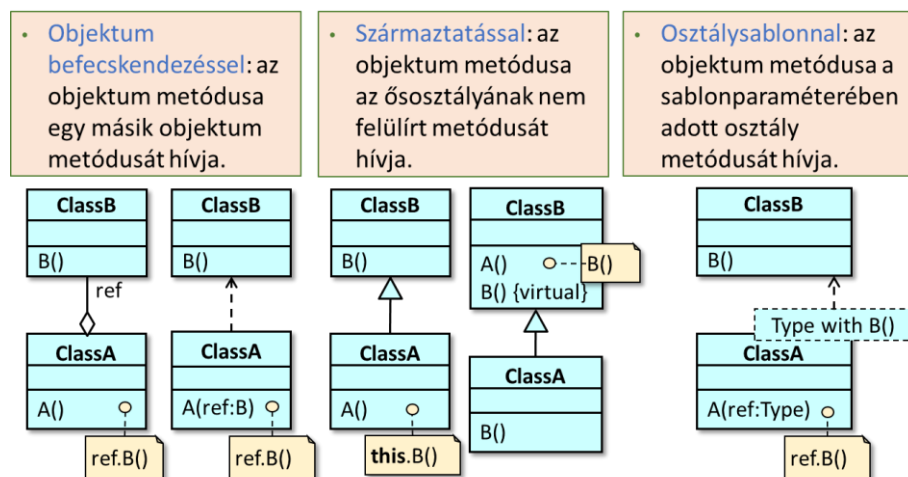
Bejáró (Iterator – viselkedési tervezési minta)

Egy gyűjtemény elemeinek felsorolását (bejárását) a gyűjteménytől elkülönülő, önálló objektummal (felsorolóval) érdemes végezni. A felsorolónak el kell tudni érni a felsorolandó gyűjteményt: hivatkozni kell tudnia rá, de nem módosítja azt. A felsoroló objektumot a felsorolni kívánt gyűjtemény hozza létre, akár többet is, és nyilván is tartja az öt bejáró felsorolót.



Felelősség átruházás, függőség befecskendezés

A tervezési minták többségében találkozunk a felelősség átruházás (dependency injection) technikájával. Erről akkor beszélünk amikor egy objektum viselkedését (metódusainak működését) nemcsak az objektum osztálya, hanem más osztály(ok) metódusai is befolyásolhatják. Ez elérhető objektum-befecskendezéssel, származtatással vagy osztálysablon alkalmazásával.



A fenti módszerek közül az első kettő objektum-orientált technika, ezekkel találkozhatunk a tervezési mintákban, amelyek némelyikének mindkét változata ismert.

A legelső módszer, az úgynevezett objektum-befecskendezés, az objektumok szintjén valósítja meg a felelősség átruházást: ugyanazon osztály két objektuma eltérő viselkedést mutathat, ha eltérő objektumokat fecskendezünk (aggregálunk) beléjük. A második módszer a származtatásra épül, és ennek megfelelően egész osztályokra (tehát egy-egy osztály összes objektumára egységesen) visz át egy másik osztályban (az őszosztályban) definiált funkcionalitást. A harmadik módszer, az osztálysablon alkalmazása nem objektum-orientált technika: szemben az előző kettővel ez fordítási időben, és nem futási időben épít be egy osztályba másik osztályban definiált funkcionalitást.